```
Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:20 # ExpenseTracker::Ledger#record with a valid expense successfully saves the expense in the DB
rspec ./spec/integration/app/ledger_spec.rb:34 # ExpenseTracker::Ledger#record when the expense lacks a payee rejects the expense as invalid

Randomized with seed 32043
```

While RSpec isn't able to tell you *why* the ordering dependency is happening, it can certainly help you identify *which* specs you need to run to reproduce it.

With the --bisect option, RSpec will systematically run different portions of your suite until it finds the smallest set that triggers a failure:

```
$ bundle exec rspec --bisect --seed 32043
Bisect started using options: "--seed 32043"
Running suite to find failures... (0.45293 seconds)
Starting bisect with 2 failing examples and 9 non-failing examples.
Checking that failure(s) are order-dependent... failure appears to be order-dependent

Round 1: bisecting over non-failing examples 1-9 . ignoring examples 1-5 (0.45132 seconds)
Round 2: bisecting over non-failing examples 6-9 . ignoring examples 6-7 (0.43739 seconds)
Round 3: bisecting over non-failing examples 8-9 . ignoring example 8 (0.43102 seconds)
Bisect complete! Reduced necessary non-failing examples from 9 to 1 in 1.64 seconds.

The minimal reproduction command is:
  rspec './spec/acceptance/expense_tracker_api_spec.rb[1:1]'
  './spec/integration/app/ledger_spec.rb[1:1:1:1,1:1:2:1]' --seed 32043
```

RSpec has given us a minimal set of specs we can run any time to see the failure:

```
$ bundle exec rspec './spec/acceptance/expense_tracker_api_spec.rb[1:1]' './spec/integration/app/ledger_spec.rb[1:1:1:1,1:1:2:1]' --seed 32043
Run options: include {:ids=>{"./spec/acceptance/expense_tracker_api_spec.rb"=>["1:1"], "./spec/integration/app/ledger_spec.rb"=>["1:1:1:1", "1:1:2:1"]}}

Randomized with seed 32043
*FF

« truncated »

Finished in 0.0485 seconds (files took 0.21859 seconds to load)
3 examples, 2 failures, 1 pending
```